Estrutura de Dados para Matrizes Esparsas: Listas Encadeadas Circulares

Carlos D. R. de Oliveira[1], **Murilo V. A. Fragoso**[2]

[1]Universidade Federal do Ceará (UFC)
Av. José de Freitas Queiroz, 5003 – 63.902-580 – Quixadá – CE – Brazil

{carlos.rodrigueszx81,murilofragoso}@alu.ufc.br

Abstract. *Sparse matrices have many applications, especially in scientific and engineering problems. A common example of sparse matrices is found in spreadsheet programs such as Excel, where a large matrix may contain only a fraction of the elements in use. In this context, spreadsheets use the matrix to store formulas, values, and strings, allocating memory space as needed, even if the matrix appears to be larger than the computer's memory capacity. This report presents an implementation of sparse matrices for double-precision floating-point data using the circular singly linked list concept, the functions created, and an analysis of the worst-case complexity of some algorithms.*

Resumo. *As matrizes esparsas têm diversas aplicações, especialmente em problemas científicos e de engenharia. Um exemplo cotidiano de aplicação de matrizes esparsas é encontrado em programas de planilha, como o Excel, onde uma matriz de grandes dimensões pode conter apenas uma fração de elementos em uso. Nesse contexto, as planilhas utilizam a matriz para armazenar fórmulas, valores e strings, alocando espaço de memória conforme necessário, mesmo que a matriz aparente ser maior que a capacidade de memória do computador. Este relatório apresenta uma implementação de matrizes esparsas para dados de ponto flutuante de precisão dupla (double) utilizando o conceito de lista simplesmente encadeada circular, as funções criadas e análise da complexidade de pior caso de alguns algoritmos.*

1. Introdução

Este trabalho tem como objetivo desenvolver e implementar um **Tipo Abstrato de Dados (TAD)** para a manipulação eficiente de **matrizes esparsas**, utilizando **listas simplesmente encadeadas circulares com nós sentinelas**. Em *Ciência da Computação*, um **TAD** é um modelo matemático que define um tipo de dado com base em seu comportamento e nas operações que podem ser realizadas sobre ele, independentemente de sua implementação interna.

A abordagem adotada neste trabalho visa **otimizar o uso de memória**, armazenando apenas os elementos diferentes de zero da matriz. Para isso, cada **linha** e cada **coluna** da matriz são representadas por *listas encadeadas circulares independentes*, permitindo operações eficientes de **inserção**, **remoção** e **consulta**. Essa estratégia segue a metodologia proposta por *Ziviani (2006)* e proporciona uma estrutura **flexível** e **escalável** para o tratamento de matrizes esparsas.

O processamento eficiente de **matrizes esparsas** é um problema de grande relevância em diversas áreas da computação, como *modelagem numérica*, *inteligência*

artificial e *bancos de dados esparsos*. Essas matrizes, caracterizadas por possuírem um grande número de **elementos nulos**, demandam abordagens específicas para reduzir o alto custo de armazenamento e processamento associado a métodos tradicionais. A manipulação *ingênua* de matrizes esparsas pode levar ao **desperdício de recursos computacionais**, tornando necessária a adoção de *estruturas de dados especializadas*.

A implementação será realizada em **C++**, seguindo as diretrizes estabelecidas na disciplina de **Estruturas de Dados** da *Universidade Federal do Ceará – Campus Quixadá*. O projeto exige a utilização de **alocação dinâmica de memória**, evitando o uso de *arrays* para garantir **maior flexibilidade** na manipulação dos dados. Além disso, operações fundamentais, como **soma** e **multiplicação** de matrizes esparsas, serão implementadas e testadas, assegurando que a solução atenda aos requisitos de **eficiência** e **escalabilidade**.

Além da implementação da estrutura de dados, este relatório apresenta uma análise detalhada das **decisões de projeto**, das **dificuldades encontradas** durante o desenvolvimento e da **complexidade computacional** das operações implementadas. A avaliação desses aspectos visa demonstrar a **eficácia** da solução proposta e sua aplicabilidade em *cenários reais*, como *processamento de planilhas*, *simulações computacionais* e *algoritmos de inteligência artificial*.

2. Motivações

Matrizes esparsas são essenciais em áreas que lidam com **grandes volumes de dados**. Em simulações numéricas de aplicações profundas da *física*, como no **método dos elementos finitos (FEM)** e na **dinâmica de fluidos computacional (CFD)**, a maioria dos elementos dessas matrizes é **nula**. Armazenar e processar apenas os valores **relevantes** reduz custos de **memória** e **tempo de execução**, viabilizando a solução de **problemas intrincados**.

Essa eficiência também é crucial em **aplicações cotidianas**. Em **planilhas eletrônicas**, como *Excel* e *Google Sheets*, a maior parte das células permanece **vazia**. Utilizar **estruturas otimizadas** para armazenar apenas *dados não nulos* torna os arquivos **menores** e o processamento **mais ágil**. Da mesma forma, em **processamento de linguagem natural (NLP)**, **matrizes esparsas** representam documentos em modelos como *bag-of-words* ou *TF-IDF*, onde a maioria das palavras **não aparece** na maioria dos textos. Sem técnicas eficientes, o manejo de **grandes volumes de dados** seria inviável.

Outro exemplo são **sistemas de recomendação**, como os da *Netflix* ou *Spotify*, que usam **matrizes esparsas** para representar interações entre usuários e itens. Como a maioria das entradas é **nula** (o usuário não interage com a maioria dos itens), a **eficiência no armazenamento e processamento** é fundamental para oferecer **recomendações em tempo real**. O mesmo ocorre em **redes sociais**, como *Facebook* e *Twitter*, onde **grafos de relacionamentos** são representados por **matrizes de adjacência extremamente esparsas**.

Diante desses cenários, fica evidente a necessidade de **estruturas de dados especializadas** para **matrizes esparsas**. A proposta deste trabalho, baseada em **listas encadeadas circulares com nós sentinelas**, busca **otimizar o armazenamento** e as **operações**, atendendo às demandas de **aplicações modernas** que exigem **eficiência no manejo de dados esparsos**.

3. Divisão de Responsabilidades

Durante o desenvolvimento deste projeto, trabalhamos de forma **colaborativa** para garantir que ninguém ficasse sobrecarregado. Grande parte do trabalho foi realizada em conjunto, muitas vezes em chamadas no *Discord*, onde discutíamos ideias, resolvíamos dúvidas e implementávamos a estrutura de dados em tempo real.

Além dessas sessões em dupla, também tivemos **autonomia** para desenvolver partes do projeto individualmente, sempre garantindo que tudo estivesse bem integrado. Para manter a organização e evitar que trabalhássemos em cima de um código defeituoso, utilizamos o *GitHub* para **alojamento** do projeto e o *Git* para **versionamento**. Isso permitiu que cada um contribuísse sem medo de perder progresso e nos ajudou a manter um *histórico claro* das mudanças realizadas.

O **relatório** foi escrito de forma simultânea, com ambos contribuindo na **documentação** e **revisão** do trabalho. Dessa forma, conseguimos garantir que todas as decisões fossem bem registradas e que o relatório refletisse fielmente o esforço conjunto no desenvolvimento do projeto.

4. Testes Executados

Para validar o correto funcionamento dos métodos `create()`, `insert()`, `get()`, `delete()` e `open()` foram realizados diversos testes considerando diferentes cenários. A seguir, apresentamos um exemplo de teste e sua saída correspondente.

4.1. Teste do comando *create*

```
> create
$ create
Erro de inicialização:linha ou coluna inválida!
```

Código 1. Comando para criar uma matriz sem os parâmetros especificados.

No exemplo acima, a tentativa de criar uma matriz sem especificar parâmetros válidos resultou em uma mensagem de erro, indicando que a linha ou coluna fornecida estava fora dos limites permitidos. Esse comportamento confirma a robustez da verificação de entrada no método implementado.

```
> create 3 3
$ create 3 3
> print 0
$ print 0
[ 0 0 0 ]
[ 0 0 0 ]
[ 0 0 0 ]
```

Código 2. Comando para criar uma matriz corretamente.

No exemplo acima, com os parâmetros corretos, a matriz foi criada corretamente. Observe que o comando *print* é necessário em nosso sistema para verificar se a matriz foi criada de fato.

4.2. Teste do comando *insert*

```
> insert
$ insert
Informe o índice da matriz corretamente!
```

Código 3. Comando para inserir um valor na matriz usado incorretamente.

No exemplo acima, sem os parâmetros corretos, a matriz não recebeu valor nenhum. Observe que o programa retorna uma mensagem em caso de erro.

```
> create 3 3
$ create
> insert 0 1 1 8
$ insert
> print 0
$ print 0
[ 8 0 0 ]
[ 0 0 0 ]
[ 0 0 0 ]
```

Código 4. Comando para inserir um valor na matriz usado corretamente.

No exemplo acima, com os parâmetros corretos, a matriz recebeu o valor passado pelo usuário. Observe que o programa precisa da função *print* para verificar se o valor foi passado para a matriz de fato.

```
> insert 0 1 1 7
$ insert
> print 0
$ print 0
[ 7 0 0 ]
[ 0 0 0 ]
[ 0 0 0 ]
```

Código 5. Comando de inserção utilizado para atualizar um valor.

4.3. Teste do comando *get*

```
> get
$ get
Informe o índice da matriz corretamente!
```

Código 6. Comando para receber um valor em índices não especificados.

No exemplo acima, a tentativa de buscar uma célula sem especificar parâmetros válidos resultou em uma mensagem de erro, indicando que a linha ou coluna ou mesmo o índice fornecido estava fora dos limites permitidos. Esse comportamento confirma a robustez da verificação de entrada no método implementado.

```
> create 3 3
$ create 3 3
> insert 0 2 2 3
$ insert 0 2 2 3
> get 0 2 2
$ get 0 2 2
Valor na célula (2, 2) é:  9
```

Código 7. Comando para buscar um valor em um índice, linha e coluna corretamente.

Observe que, no exemplo acima, o resultado de buscar uma célula com parâmetros válidos precisou que antes uma matriz fosse criada corretamente, indicando a necessidade de uma sequência de comandos fornecidos corretamente. Esse comportamento confirma o encadeamento regrado de entrada no método implementado.

4.4. Teste do comando *open*

```
> open a_path
$ open a_path
Arquivo inválido ou não encontrado: a_path
> open non_matrix.txt
$ open non_matrix.txt
Erro de inicialização:  linha ou coluna inválida!
```

Código 8. Comando para ler uma matriz a partir de um arquivo inexistente/não encontrado.

No exemplo acima, ao tentar abrir um arquivo que não existe ou é inválido o programa imprime uma mensagem de erro e o arquivo não é criado, no entanto a execução do programa continua.

```
> open tests/matrix4x4.txt
$ open tests/matrix4x4.txt
Arquivo carregado!
> show
$ show
matriz[0]: 4x4
```

Código 9. Comando para ler uma matriz a partir de um arquivo existente/válido.

Neste outro exemplo, poder ser observado que ao ler um arquivo existente e válido a mensagem que é impressa é justamente a que confirma a leitura do arquivo e a criação da matriz nova matriz.

4.5. Teste do comando *delete*

```
> create 3 3
$ create 3 3
```

```
> show
$ show
matriz[0]: 3x3
> delete
$ delete
Informe o índice da matriz corretamente!
```

Código 10. Comando para deletar um índice incorreto ou não especificado.

No exemplo acima, a tentativa de deletar uma matriz sem especificar o parâmetro válido resultou em uma mensagem de erro, indicando que o índice fornecido estava fora dos limites permitidos. Esse comportamento confirma a robustez da verificação de entrada no método implementado.

```
> create 3 3
$ create 3 3
> print 0
$ print 0
[ 0 0 0 ]
[ 0 0 0 ]
[ 0 0 0 ]
> delete 0
$ delete 0
Matriz de índice 0 deletada!
> print 0
$ print 0
Nenhuma matriz encontrada no índice 0!
```

Código 11. Comando para deletar uma matriz corretamente.

No exemplo acima, o resultado de deletar uma matriz com parâmetro válido retornou uma mensagem de que o comando foi bem-sucedido, indicando que o índice fornecido estava dentro dos limites. Esse comportamento confirma a robustez da verificação de entrada no método implementado. Lembre-se de que os comandos anteriores também devem ser corretos.

5. Dificuldades Encontradas

Inicialmente, não identificamos dificuldades significativas no desenvolvimento do projeto. No entanto, à medida que avançávamos na implementação, percebemos a necessidade de funções auxiliares para garantir a correta execução das operações previstas.

Um exemplo disso foi a implementação do destrutor da matriz esparsa circular. Para assegurar a liberação adequada da memória alocada dinamicamente, foi necessário desenvolver uma função auxiliar `clear`, responsável por percorrer toda a estrutura e desalocar os nós corretamente.

Além disso, durante a implementação das operações de soma e multiplicação de matrizes, identificamos a necessidade de um mecanismo eficiente para armazenar a matriz resultante. Para isso, realizamos a sobrecarga do operador de atribuição (=), permitindo

que a matriz resultante fosse corretamente armazenada no vetor, garantindo a integridade dos dados e evitando a necessidade de cópias manuais desnecessárias.

Esses desafios exigiram ajustes na arquitetura do código e reforçaram a importância do planejamento modular, garantindo que cada operação fosse implementada de forma clara e reutilizável.

6. Análise de Complexidade

Para esta análise, será necessário avaliar o tempo de execução do algoritmo, determinando o número de passos a serem executados para uma entrada n. Utilizaremos a notação assintótica, especificamente o *Big-O*, que representa um limite superior para o crescimento da função. Essa notação classifica algoritmos com base em seu desempenho no pior, melhor e caso médio, independentemente da máquina utilizada. Neste relatório, aplicaremos esse conceito para calcular a complexidade de alguns algoritmos implementados, focando apenas no pior caso.

6.1. Método insert()

O funcionamento desse método depende do estado atual da matriz para a qual foi chamado. Caso não exista uma célula nos índices i e j fornecidos como parâmetro, o método cria um nó que representa uma célula e armazena os valores de entrada, os índices i e j, e um valor do tipo *double*. Nesse caso, o método ajusta a posição do novo nó em relação aos demais nós. Se, por outro lado, já existir uma célula nos índices especificados, o método é responsável por atualizar o valor dessa célula com o novo valor fornecido. Além disso, o método também se encarrega de remover a célula especificada caso o valor passado como parâmetro seja igual a 0. No Código 12 é possível ver sua implementação.

```
1  void SparseMatrix::insert(int i, int j, double value) {
2    if (i<=0 || i>linhas || j<=0 || j>colunas) {
3      throw std::out_of_range("Linha/coluna fora do intervalo");
4    }
5    Node* aux = m_head->direito;
6    while (aux->coluna!=j) { aux = aux->direito; }
7    while (aux->abaixo->linha != 0 && aux->abaixo->linha<i) {
8      aux = aux->abaixo;
9    }
10   if (aux->abaixo->linha==i) {
11     if (value!=0) {
12       aux->abaixo->valor = value;
13     } else {
14       Node* toErase = aux->abaixo;
15       aux->abaixo = toErase->abaixo;
16       aux = m_head->abaixo;
17       while (aux->linha!=i) { aux = aux->abaixo; }
18       while (aux->direito->coluna!=0 && aux->direito->coluna<j){
19         aux = aux->direito;
20       }
21       aux->direito = toErase->direito;
22       delete toErase;
23     }
```

```
24      return;
25    }
26    Node* temp = new Node(i, j, value);
27    temp->abaixo = aux->abaixo;
28    aux->abaixo = temp;
29    aux = m_head->abaixo;
30    while (aux->linha!=i) { aux = aux->abaixo; }
31    while (aux->direito->coluna!=0 && aux->direito->coluna<j) {
32      aux = aux->direito;
33    }
34    temp->direito = aux->direito;
35    aux->direito = temp;
36  }
```

Código 12. Método para inserir um valor na matriz.

Note que os laços **while**, que buscam pela coluna j e pela linha i, são as partes mais importantes do algoritmo, pois determinam se a posição a ser inserida já existe ou não. Essas linhas são sempre executadas quando o método é chamado.

No pior caso do algoritmo, a busca pela coluna j e pela linha i percorre toda a lista de colunas e linhas, respectivamente. Isso acontece quando a coluna j está na última posição da lista de colunas e a linha i está na última posição da lista de linhas não nulas. Portanto, a entrada de pior caso ocorre quando a matriz esparsa está completamente preenchida ou quando estamos tentando inserir em uma posição que está no final da lista.

Para determinar a função $T(n)$ que representa o número de passos do algoritmo *insert* para uma entrada de tamanho n, precisamos contar quantas vezes as linhas que realizam as buscas são executadas, note que caso a condição da linha 10 seja verdadeira o algoritmo fora da condição não será executado e caso seja falsa o algoritmo dentro do condicional não será. A princípio suponha que a os valores i e j são iguais a um valor $n \in \mathbb{Z}$.

A linha 6, que busca pela coluna j, percorre a lista de colunas. No pior caso, isso requer que ela seja executada $n-1$ vezes. A linha 7, que busca pela linha i, percorre a lista de linhas. No pior caso, é executada $n-1$ vezes. Se a condição da linha 10 é verdadeira, a posição já existe e a atualização do valor ou a remoção do nó é executada em tempo constante, mas a busca pela linha e pela coluna são executadas, assim como antes, $n-1$ vezes cada.

Se a condição for falsa, a posição não existe e a inserção de um novo nó requer novamente as buscas pela linha e pela coluna, e portanto, a linha 30 é executada $n-1$ e a linha 31 é executada $n-1$ vezes novamente. Assim, somando as operações que **serão executadas**, temos que $T(n) = (n-1) + (n-1) + (n-1) + (n-1) = 4(n-1)$, logo a complexidade de tempo no pior caso é $T(n) = 4(n-1)$, que é uma função linear. Utilizando os conceitos de notação assintótica, conclui-se que $T(n) = 4(n-1) = O(n)$.

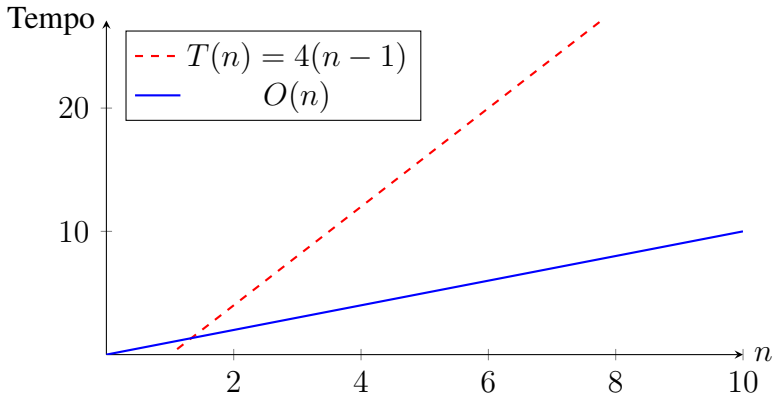


Gráfico 1. Gráfico das funções $T(n) = 4(n-1)$ e $O(n)$ para $n \geq 0$

6.2. Método get()

Esse método percorre as linhas e colunas da matriz que a chamou até a linha i e coluna j especificados, caso o método encontre um valor na posição passada como parâmetro do método, ela o retorna, caso contrário retorna 0. No Código 13 é possível ver sua implementação.

```
1  double SparseMatrix::get(int i, int j) const {
2    if (i<=0 || i>linhas || j<=0 || j>colunas) {
3      throw std::out_of_range("Linha/coluna fora do intervalo");
4    }
5    Node* aux_linha = m_head->abaixo;
6    while (aux_linha->linha!=m_head->linha && aux_linha->linha<i)
7      aux_linha = aux_linha->abaixo;
8
9    if (aux_linha==m_head || aux_linha->linha!=i) return 0;
10   Node* aux_coluna = aux_linha->direito;
11   while (aux_coluna!=aux_linha && aux_coluna->coluna<j)
12     aux_coluna = aux_coluna->direito;
13
14   if (aux_coluna==aux_linha || aux_coluna->coluna!=j) return 0;
15   return aux_coluna->valor;
16 }
```

Código 13. Método para retornar um valor da matriz.

Note que os laços **while** nas linhas 6 e 11, que buscam pela linha i e coluna j, respectivamente, são os principais nesse algoritmo, eles sempre serão executados ao chamar o método em um tempo que varia de acordo com o tamanho da entrada. Para determinar a função $T(n)$ que dá o número de passos desse algoritmo, será necessário contar a quantidade de vezes que tais laços são executados. Suponha que os valores de i e j são iguais a um valor $n \in \mathbb{Z}$.

O laço na linha 6 percorre todas as linhas da matriz até a linha especificada n. No pior caso, quando n é igual ao número total de linhas da matriz, o laço é executado $n-1$ vezes. Na linha 11, o laço percorre todas as colunas, exceto a coluna inicial. O pior caso

ocorre quando n corresponde ao número total de colunas da matriz e todas as colunas na linha especificada possuem valor, resultando também em $n - 1$ execuções.

Assim, somando as operações a serem executadas, temos que $T(n) = (n - 1) + (n - 1) = 2(n - 1)$. Portanto, o tempo de execução no pior caso é $T(n) = 2(n - 1)$, que é uma função linear. Conclui-se que $T(n) = 2(n - 1) = O(n)$, utilizando a notação assintótica.

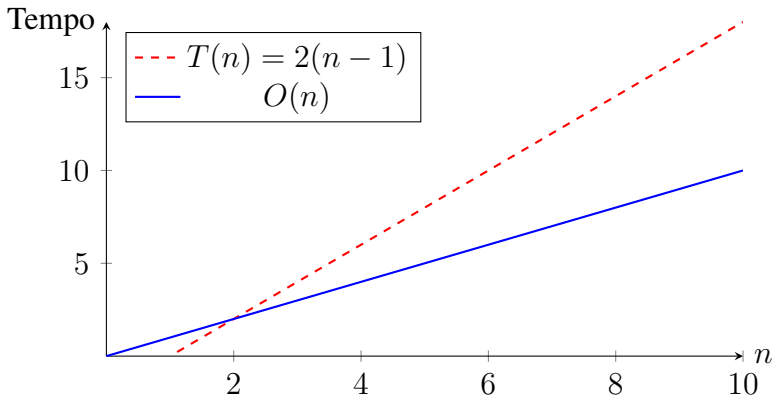


Gráfico 2. Gráfico das funções $T(n) = 2(n - 1)$ e $O(n)$ para $n \geq 0$

6.3. Função sum()

Essa função é responsável por somar duas matrizes passadas por referência. Ela retorna uma nova matriz esparsa *C* com o resultado da soma entre as duas matrizes de entrada. No entanto, caso as matrizes de entrada tenham colunas ou linhas diferentes uma da outra, a função lança uma exceção. No Código 14 é possível ver sua implementação.

```
1  SparseMatrix *sum(SparseMatrix &A, SparseMatrix &B) {
2    if (A.getCols()!=B.getCols() || A.getLines()!=B.getLines()){
3      throw out_of_range("Linhas/colunas distintas");
4    }
5
6    SparseMatrix *C = new SparseMatrix(A.getLines(), A.getCols());
7    for (int i=1; i <= C->getLines(); i++){
8      for (int j=1; j <= C->getCols(); j++){
9        C->insert(i, j, (A.get(i, j) + B.get(i, j)));
10     }
11   }
12   return C;
13 }
```

Código 14. Função para somar duas matrizes esparsas.

Note que a linha 9 é responsável por inserir e acessar elementos das matrizes que serão somadas. Logo, essa é a linha mais importante desse algoritmo e que é sempre executada quando entramos nos dois laços **for** aninhados. O pior caso desse algoritmo é justamente quando uma das duas matrizes a serem somadas estão preenchidas com valores sendo necessário acessar todos os seus elementos para realaizar a operação.

Portanto, a fim de determinar a função $T(n)$, é preciso que seja contado quantas vezes essa linha é executada. Note que o laço **for** da linha 7 percorre a quantidade de linhas na matriz C iniciando de $i = 1$, o da linha 8 percorre a quantidade de colunas da mesma matriz iniciando de $j = 1$. Suponha que a quantidade de linhas e colunas da matriz C são iguais a um número $n \in \mathbb{Z}$. Assim, perceba que o **for** da linha 7 e da linha 8 serão executado n vezes cada e por consequência as operações feitas na linha 9 serão executadas $n \times n$ vezes. Sabe-se que o método insert() e o método get() possuem tempo de execução $4(n-1)$ e $2(n-1)$, respectivamente. Logo, temos que

$$
\begin{aligned}
T(n) &= n^2 \times [4(n-1) + 2(n-1)] \\
T(n) &= n^2 \times 6(n-1) \\
T(n) &= 6n^2(n-1) \\
T(n) &= 6n^3 - 6n^2
\end{aligned}
$$

Assim, a complexidade de pior caso é $T(n) = 6n^3 - 6n^2$, que é uma função cúbica onde o termo dominante, à medida que o valor de n cresce, é $6n^3$. Utilizando a notação assintótica, temos que $T(n) = 6n^3 - 6n^2 = O(n^3)$.

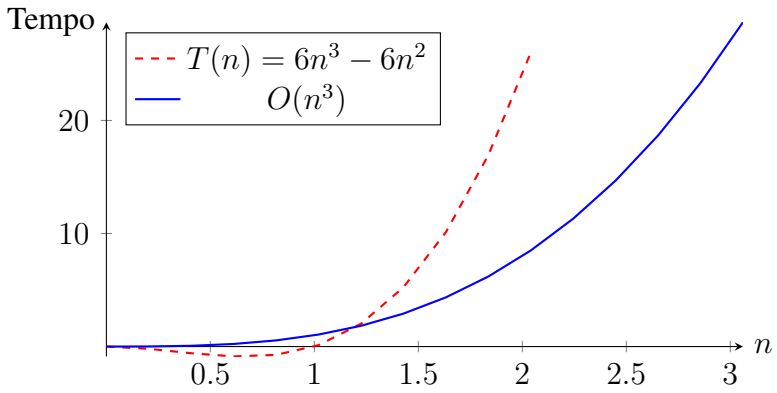


Gráfico 3. Gráfico das funções $T(n) = 6n^3 - 6n^2$ e $O(n^3)$ para $n \geq 0$

References

[1] LUIS, A. *Exemplo de interface para o Projeto de ED 2024.2*. YouTube, 2024. Disponível em: https://www.youtube.com/watch?v=rwklzd6Zpoc&t=758s. Acesso em: 07 fev. 2025.

[2] ZIVIANI, N.; BOTELHO, F. C. *Projeto de Algoritmos com Implementações em Java e C++*. São Paulo: Cengage Learning, 2006.

[3] C++ REFERENCE. *The C++ Resources Network*. Disponível em: https://en.cppreference.com/w/. Acesso em: 07 fev. 2025.

[4] BRASIL ESCOLA. *Multiplicação de Matrizes*. Disponível em: https://brasilescola.uol.com.br/matematica/multiplicacao-matrizes.htm. Acesso em: 07 fev. 2025.

[5] BRASIL ESCOLA. *Adição e Subtração de Matrizes*. Disponível em: https://brasilescola.uol.com.br/matematica/adicao-subtracao-matrizes.htm. Acesso em: 07 fev. 2025.

[6] TABNEWS. *Análise do Tempo de Execução de Algoritmos para Iniciantes*. Disponível em: https://www.tabnews.com.br/ritacarolina/analise-do-tempo-de-execucao-de-algoritmos-para-iniciantes. Acesso em: 07 fev. 2025.

[7] IME-USP. *Análise de Algoritmos e Notação O Grande*. Disponível em: https://www.ime.usp.br/~pf/analise_de_algoritmos/aulas/Oh.html. Acesso em: 07 fev. 2025.

[8] WIKIPÉDIA. *Tipo abstrato de dado*. Disponível em: https://pt.wikipedia.org/wiki/Tipo_abstrato_de_dado#:~:text=Na%20ci%C3%AAncia%20da%20computa%C3%A7%C3%A3o%2C%20um,objetos%2C%20que%20%C3%A9%20definido%20pelo. Acesso em: 07 fev. 2025.

[9] OVERLEAF. *Learn LaTeX in 30 minutes*. Disponível em: https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes. Acesso em: 09 fev. 2025.